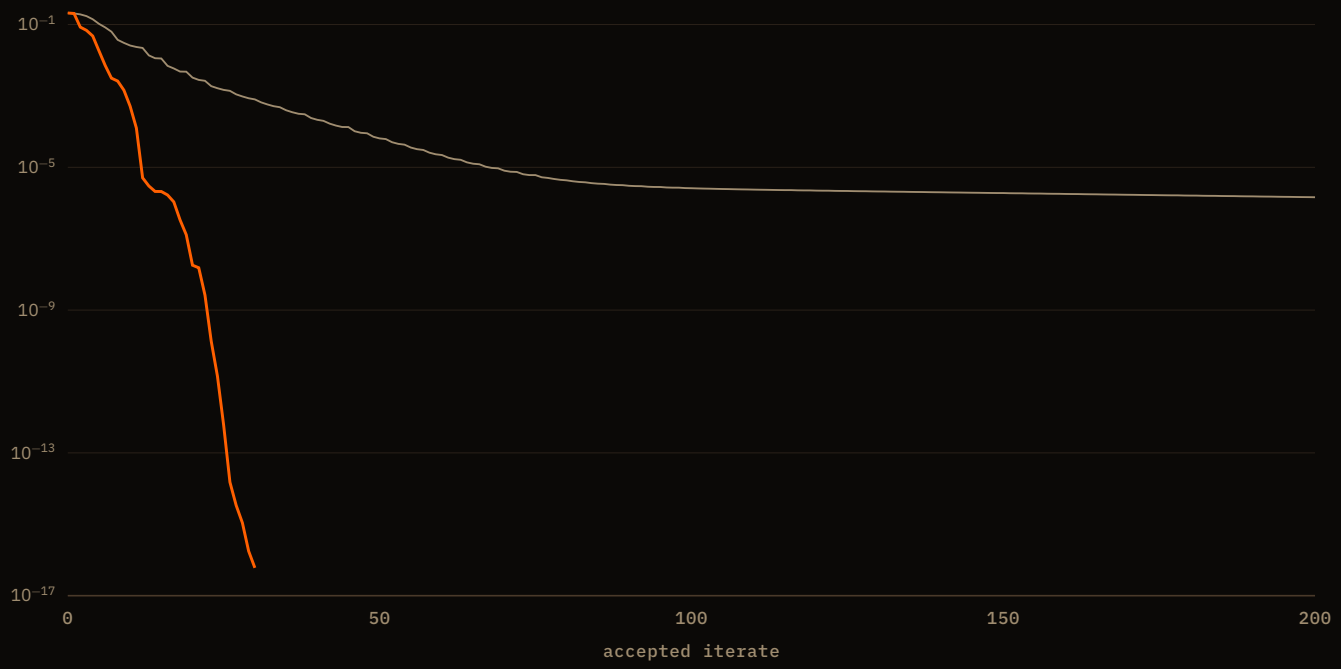



FIVE CAD PARAMETERS OF A FILLETED, DRILLED BOX · OBJECTIVE PER ITERATE — L-BFGS ON EXACT BREP GRADIENTS

# Exact Gradients Through a BRep Kernel

Differentiating CAD parameters through tessellation, fillets, mass properties and contact

CAM PEDERSEN · MUNICIPAL ROBOTICS

## Abstract

Gradient-based design needs the derivative of a downstream quantity with respect to the parameters a designer actually edits: a hole radius, a fillet radius, an extrude depth. In a boundary-representation (BRep) CAD kernel those parameters pass through Boolean operations, filleting and tessellation, all of which make discrete choices, so the kernel is not differentiable as written. We describe the differentiable seam in vcad, an open-source Rust BRep kernel: rather than differentiating the kernel's combinatorics, we freeze the tessellation's topology at the current parameter value and differentiate the positions of its nodes exactly, by evaluating the kernel's analytic surfaces on dual numbers and by implicit differentiation of the surface equations at vertices and trim curves. The seam detects changes of topology and reports them as errors instead of returning wrong gradients.

The seam covers planes, cylinders, spheres, cones and tori, including composite motions such as a fillet radius that moves twenty surfaces at once; exact first and directional second derivatives of volume, mass, centroid and inertia; a reverse-mode transpose that prices every parameter from one pass; seeding synthesized from the build function; L-BFGS; gradients of a named parameter of a `.vcad` document; and, through an exact trajectory adjoint in the phyz simulator, gradients of rigid-body rollouts including penalty contact against the CAD surface. Every derivative is validated against finite differences of a full rebuild. Fillet-radius volume gradients agree to $9.5 \times 10^{-9}$, reverse mode prices both parameters of a filleted cube from one pass and matches forward mode to rounding, and a contact rollout's shape gradient to $3.5 \times 10^{-7}$. L-BFGS recovers all five parameters of a filleted, drilled box in 30 iterations where projected gradient descent does not converge in 200. The gradients are those of the frozen tessellation: they are exact for the mesh the kernel produces and differ from the continuum shape derivative by the polygonization error, and they say nothing across a change of topology. We present the seam as one operator with its own adjoint, the geometry node in a planned architecture in which vcad's physics is written once as math and the tang compiler derives kernels and gradients for every device; today the seam, mass properties and the rollout adjoint are composed by hand, and that composition is the graph the architecture will make general.

## 1 Introduction

A mechanical designer works in parameters: the depth of a pocket, the radius of a bore, the radius of a fillet. An optimizer that should help that designer needs $dJ/d\theta$, the derivative of an objective $J$ (a mass, an inertia, a simulated trajectory) with respect to those same parameters. Production CAD kernels do not provide it. Between $\theta$ and anything measurable sit Boolean operations, fillet construction and tessellation, each of which makes discrete decisions: which faces survive a Boolean, in which order they are sewn, how many segments approximate a circle, where a triangulation inserts a vertex. Practitioners fall back on finite differences of full rebuilds, which cost one rebuild per parameter and, as we show, are noisy on filleted parts, because two rebuilds can enumerate the same topology in different orders.

Most differentiable-geometry work avoids the problem by changing the representation: signed distance fields, constructive solid geometry over smooth primitives, or meshes optimized directly. These are well suited to learning and to rendering, but the designer's model is a BRep with fillets, and its parameters are the ones in the feature tree.

We take the opposite approach and differentiate the production kernel without changing it. The idea, which we call the **seam**, is to cut the chain at the tessellated mesh. At the current parameter $\theta_0$ we record a **frozen plan**: the tessellation's connectivity and, for every mesh node, a recipe that says how that node is determined by the BRep (a topology vertex, a point at fixed surface coordinates, or a point on the intersection of two surfaces). Within the neighbourhood where the topology does not change, node positions are smooth functions of $\theta$, and we compute their derivatives $dx_i/d\theta$ exactly. Everything downstream (mass properties, objectives, a physics adjoint) is an ordinary smooth function of node positions.

Our contributions are:

- **A frozen-tessellation seam for a production BRep kernel** (vcad), with exact node sensitivities from dual-number evaluation of the kernel's analytic surfaces and implicit differentiation of surface equations at vertices and trims, covering planes, cylinders, spheres, cones and tori, and composite motions such as a fillet radius (Section 3).
- **Exact derivatives of mass properties**, first order and directional second order including node accelerations on cones and tori, and a Gauss–Newton Hessian for least-squares objectives (Section 4).
- **A reverse mode** obtained by transposing the seam's per-node linear maps, so one pass prices every parameter, and an L-BFGS optimizer over it that treats topology changes as failed steps (Section 5).
- **Seeding synthesized from the build function**, which removes the last hand-written step, and gradients of named parameters of `.vcad` documents (Section 6).
- **Gradients through rigid-body rollouts**, exact for contact-free rollouts and for a penalty-contact model whose contact cotangent on the CAD surface is pulled back through the seam (Section 7).

Every derivative is validated against central finite differences of a full CAD rebuild (Section 9), and a script regenerates every number in this paper from a fresh clone (Section 11).

The seam also fits an operator-graph design. It takes CAD parameters and returns a mesh together with a vector–Jacobian product, which is the contract of an opaque operator node in a differentiable program. Section 8 places it in the architecture we are building for vcad and phyz on the tang compiler, where engineering physics is written once as tensor comprehensions and gradients through solves and rollouts come from operator nodes with known adjoints. The results in this paper are for the seam and its hand-wired consumers as they exist today; the architecture is stated as the plan they plug into, not as a result.

## 1.1 Scope and limits

We state the limits first, because they define what the gradients mean.

- **Gradients of the frozen tessellation.** The seam differentiates the mesh the kernel actually produces at $\theta_0$, with its segment counts and triangulation held fixed. It is exact for that mesh. It differs from the continuum shape derivative by the polygonization error: for a 112-segment hole the gap is $5.2 \times 10^{-4}$, and for 16-segment fillet blends 6.7% (Table 1). Refining the mesh closes this gap; the seam does not.
- **No derivatives across topology changes.** When a perturbation would change the topology (a blind hole breaking through, a fillet consuming a face), the derivative of the frozen mesh does not exist in any useful sense. The seam detects this with a topology signature and per-node correspondence checks and returns an error. The optimizer treats such a step as failed and shrinks it.
- **The contact adjoint covers phyz's differentiable penalty model**, a per-vertex spring–damper against a ground plane with no friction. It does not differentiate the GJK/EPA collision pipeline used by vcad's production physics world[1].
- **Round-off agreement is on CPU in f64.** Every gate in this paper runs on CPU in double precision. No GPU result is claimed.
- **The trajectory adjoint supports single-degree-of-freedom joints** (revolute, prismatic, fixed), open-loop control and objectives on the final state only.

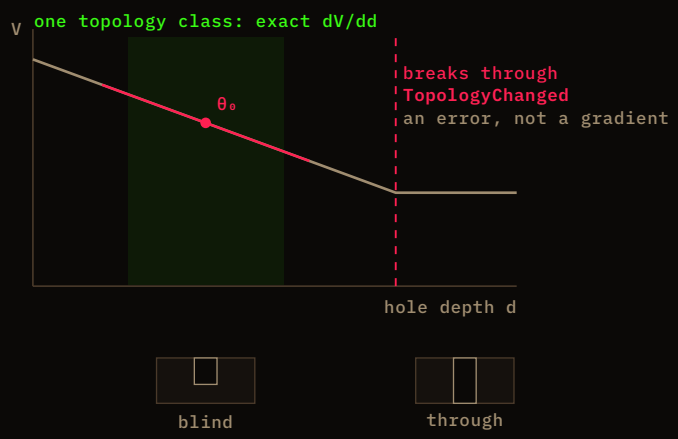



**FIG 1 What the gradient is a gradient of.** Within one topology class the frozen plan's volume is smooth in the parameter and the seam returns its exact derivative (red tangent). When a blind hole breaks through, the plan's topology signature changes and the seam returns `TopologyChanged`; an optimizer treats that as a failed step and shrinks it. Schematic.

## 2 Background and related work

**Differentiable rendering and mesh optimization.** Differentiable rasterizers[2,3] and path tracers[4] compute derivatives of images with respect to mesh vertices. They operate on a mesh whose connectivity is held fixed, as ours is; the difference is where the vertex positions come from. Worchel and Alexa extract a mesh from parametric curves and surfaces and pass it to a differentiable mesh renderer[5]; they report that naive differentiable tessellation is fragile, which our correspondence machinery addresses for BRep solids.

**Implicit and CSG representations.** Signed distance fields, learned[6] or analytic, are differentiable almost everywhere and can be rendered differentiably[7]. Program-synthesis approaches recover CSG trees from shapes[8,9]. Commercial implicit-modeling optimizers work in the same representation (TODO: find a citable reference for a production implicit-modeling optimizer). These representations trade the BRep's exact faces, trims and fillets for differentiability; we keep the BRep.

**Differentiable CAD programs.** Cascaval et al. write CAD programs in an automatically differentiable language for bidirectional editing; they do not optimize across changes of topology[10]. Michel and Boubekeur compute parameter updates for direct manipulation of procedural shapes[11]. Schulz et al. precompute a design space of CAD parameters for interactive optimization[12]. The closest work in engineering is the algorithmic differentiation of the Open CASCADE kernel with ADOL-C by Banović et al., coupled to an adjoint CFD solver[13]. They differentiate the kernel's operations by operator overloading; we leave the kernel's combinatorial code undifferentiated and differentiate only the geometric map from parameters to the frozen mesh, which is why a nondeterministic Boolean or fillet is not a problem for us.

**CAD-parameterized shape optimization.** Structural shape optimization has long used design velocities, the sensitivity of boundary points to design variables[14,15]. Agarwal et al. compute parametric design velocities for CAD models for use with adjoint solvers[16], and isogeometric analysis removes the mesh by computing on

the CAD basis[17]. Our node velocities $dx_i/d\theta$ are design velocities; what we add is computing them exactly inside a general-purpose BRep kernel, including fillets, and carrying them to mass properties and rigid-body dynamics.

**Differentiable physics and co-design.** Differentiable simulators include rigid-body formulations with contact[18–20] and differentiable programming systems[21]. Robot co-design work optimizes morphology and control jointly[22,23], usually over parametric primitives rather than CAD models. Our physics consumer is a discrete adjoint over Featherstone's articulated-body algorithm[24]; the contribution here is the interface, which prices CAD parameters through two channels, the ten mass-property scalars per body and the contact cotangent on the CAD surface.

**Differentiable array compilers.** The architecture in Section 8 builds on tang, our tensor compiler, whose single compute primitive is the comprehension $\text{out}[i] = \bigoplus_k \text{body}(x[\text{view}(i,k)])$ with a traced scalar body, a monoid combine and affine, guarded or gather views; tang derives gradients of comprehension programs and generates code for CUDA, Metal, WGSL, OpenCL, GLSL and C, checked against an f64 reference interpreter (TODO: cite the tang paper once it has an arXiv identifier). Operator nodes with user-supplied adjoints, such as a linear solve differentiated by the implicit function theorem, are the standard way to place solvers inside such programs (TODO: cite implicit-differentiation libraries, e.g. JAX custom VJPs, after verifying a reference).

## 3 The differentiable seam

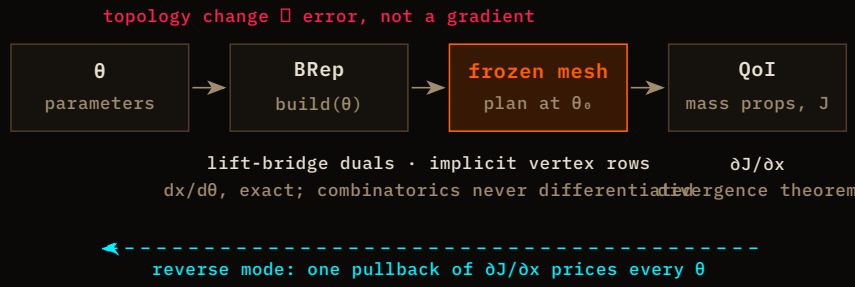



**FIG 2 Where the chain is cut.** The BRep build is never differentiated. At the current parameters a frozen plan records the tessellation; node sensitivities $dx/d\theta$ come from the kernel's analytic surfaces. Downstream quantities differentiate as smooth functions of node positions, and reverse mode runs the same maps transposed.

### 3.1 Frozen plans

vcad's kernel represents solids as BReps whose faces carry analytic surfaces (plane, cylinder, cone, sphere, torus, bilinear patch, B-spline), Boolean operations, filleting and a tessellator. Two properties of the kernel, both found by running it, shaped the design. First, the Boolean pipeline and the fillet kernel are order-nondeterministic: two builds of the same model can enumerate isomorphic topology in different orders, flip a blend cylinder's axis or rotate its reference direction. Traversal order therefore cannot carry node identity from one build to another. Second, the stock mesh stores 32-bit positions, at which a central difference with $h = 10^{-6}$ is meaningless, so the seam emits its own 64-bit mesh.

`capture_plan` tessellates once at $\theta_0$ and classifies every node:

- **TopoVertex**: the node coincides with a topology vertex. Its velocity comes from implicit differentiation (Section 3.3).
- **SurfaceUv**: the node is interior to a face, stored as fixed surface coordinates $(u, v)$. Its position is $S(u, v; \varphi(\theta))$ for the surface's fields $\varphi$ (offset, radius, axis, half-angle).
- **Boundary**: the node lies on the intersection of two faces with independent normals but is not a topology vertex, for example a ring on a cylinder's cap. Its position is the Newton solution of $\{g_{a(x)} = 0, g_{b(x)} = 0, t \cdot (x - x_0) = 0\}$, with $t$ the intersection tangent, so the node follows the moving trim whichever surface carries $\theta$.

A plan carries a topology signature (a sorted multiset of per-face descriptors, which is invariant to isomorphic reordering) and each node's capture-time position. Re-evaluating at a nearby $\theta$ resolves vertices and faces geometrically, by nearest match after transporting the stored $(u, v)$ through the rebuilt surface's frame, and fails with `CorrespondenceLost` when any node lands more than $10^{-3}$ mm from where it should. A changed signature fails with `TopologyChanged`. Both are errors, never a mesh.

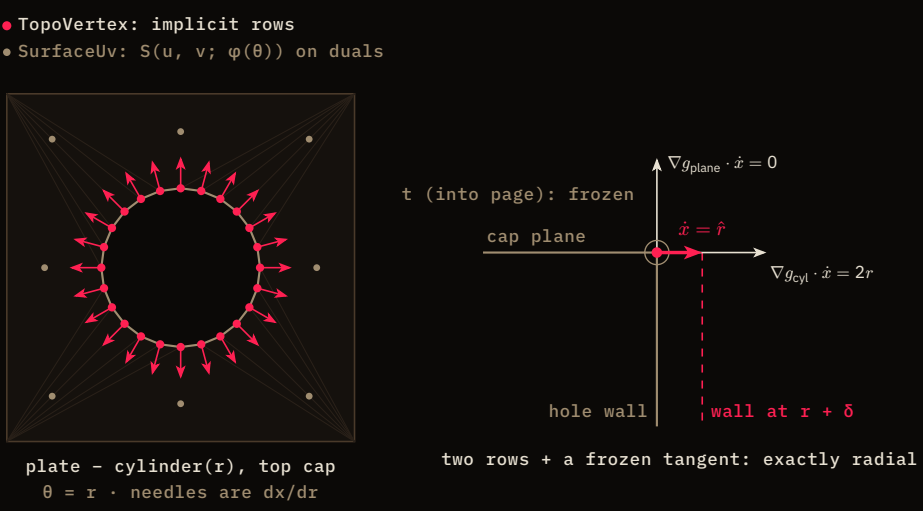



**FIG 3 Node recipes on a Boolean result.** Left: the top cap of a plate minus a cylinder, θ = hole radius. Rim vertices are topology vertices; their velocity comes from the implicit rows and is exactly radial. Plane-interior samples sit on a surface that does not move and have zero velocity. Right: the rows at one rim vertex. The cap plane and the hole wall give two independent rows; the tangent direction is unconstrained and stays frozen, so the rim keeps its angular parameter.

## 3.2 Surface nodes: the lift bridge

Every analytic surface in the kernel is generic over a scalar type, so it can be evaluated on dual numbers $a + b\varepsilon$ with $\varepsilon^2 = 0$[25]. A parameter is connected to the surfaces it moves by a **seeding**: a map from surface index to field velocities (translate by $\boldsymbol{v}$, grow the radius at rate $\dot{r}$, open the cone at rate $\dot{\alpha}$, and so on). Seeds compose additively, which matters for fillets: when the fillet radius grows, each blend cylinder's radius grows at rate 1 while its axis retreats from the edge, and each corner sphere does the same in three coordinates. For a SurfaceUv node, the dual part of $S(u, v; \varphi + \dot{\varphi}\varepsilon)$ is exactly $dx/d\theta$.

### 3.3 Vertices and trims: implicit differentiation

A topology vertex lies on every surface incident to it. Each incident surface has an implicit form $g(x;\varphi)=0$: for a plane $n\cdot(x-o)$, a cylinder $\|x_\perp\|^2-r^2$, a sphere $\|x-c\|^2-r^2$, and for a cone with apex $a$, axis $d$ and half-angle $\alpha$,

$$g=\|x_\perp\|^2-\tan^2\alpha((x-a)\cdot d)^2.$$

Differentiating $g(x(\theta);\varphi(\theta))=0$ gives one linear row per surface,

$$\nabla g\cdot\dot{x}=-\frac{\partial g}{\partial\theta}.$$

We solve the rows by Gram–Schmidt, carrying the right-hand side. Independent rows fix components of $\dot{x}$, dependent rows must agree within tolerance (otherwise the solve fails), and directions no row constrains stay at zero. The last rule is the frozen-branch convention: a node on a circular rim keeps its angular parameter. Three planes at a box corner give a full $3\times3$ solve; a plane and a cylinder at a hole rim give a rank-2 solve with a frozen tangent.

The rows needed two refinements. Boolean sewing does not always pair the half-edges around a rim, so topological adjacency under-constrains some vertices, and the seam also collects rows by geometric incidence. And a fillet blend touches its support face **tangentially**, so the blend's row is parallel to the plane's and the system is rank-deficient exactly where the corner vertex slides along the face. The missing information is that the vertex must stay on the tangent curve; for a cylinder of axis $a$ tangent to a plane of normal $n$ we add the row $q\cdot\dot{x}=q\cdot\dot{v}_{\text{translate}}$ with $q=a\times n$, and two such rows for a sphere. With these rows the corner of a filleted cube slides at exactly $(1,1,0)$ per unit radius.

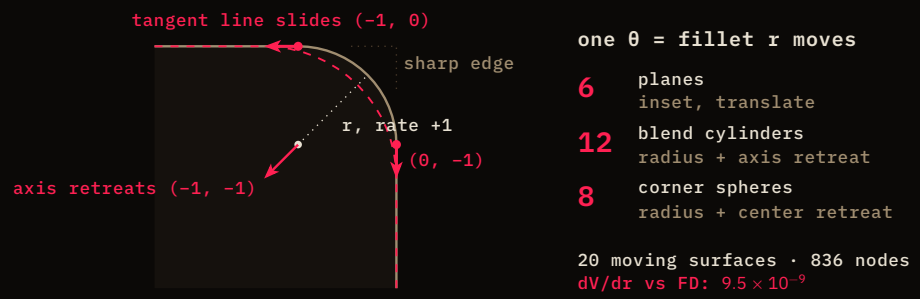



**FIG 4 A fillet radius is a composite seed.** Growing the radius by δ grows each blend cylinder's radius at rate 1 while its axis retreats at $(-1,-1)$ in the section plane; the two motions compose so that the tangent lines slide along the support faces at exactly unit rate. On the all-edges-filleted cube one parameter moves 20 surfaces this way.

## 4 Quantities of interest

### 4.1 Mass properties

Volume, mass, centroid and inertia tensor follow from signed-tetrahedron integrals over the closed mesh[26]. The integrals are written once over a generic scalar, so evaluating them at dual-number node positions gives every quantity and its derivative along $\theta$ in one pass. A physics functional plugs in through the contraction

$$\frac{dJ}{d\theta} = \sum_i \frac{\partial J}{\partial x_i} \cdot \frac{dx_i}{d\theta},$$

with $\partial V/\partial x_i$ known in closed form from the divergence theorem.

### 4.2 Second order

For one direction in parameter space,

$$\frac{d^2V}{d\theta^2} = \sum_{ij} \frac{\partial^2 V}{\partial x_i \partial x_j} \dot{x}_i \dot{x}_j + \sum_i \frac{\partial V}{\partial x_i} \ddot{x}_i.$$

Evaluating the volume integral on nested duals $((x, \dot{x}), (\dot{x}, \ddot{x}))$ carries both terms. The work is in the node accelerations $\ddot{x}$. A surface node on a plane, cylinder or sphere is linear in its fields, so its acceleration is the first-order lift fed the field accelerations. Cone and torus points are not linear in their fields ($\tan\alpha$, the two torus radii), so surface nodes are lifted on nested duals. Vertex and trim nodes solve the implicit system differentiated twice,

$$\nabla g \cdot \ddot{x} = -\left( \frac{\partial^2 g}{\partial \theta^2} + 2\dot{x}^\top \nabla_x \frac{\partial g}{\partial \theta} + \dot{x}^\top \nabla^2 g \, \dot{x} \right),$$

with the same row gradients as the first-order solve; the cone and torus contribute closed-form curvature terms because their $\nabla^2 g$ is not constant. For least-squares objectives $J = \sum_q r_q^2$ we also provide the Gauss–Newton Hessian $2J_r^\top J_r$, which drops the residual-curvature term and is therefore exact at zero residual.

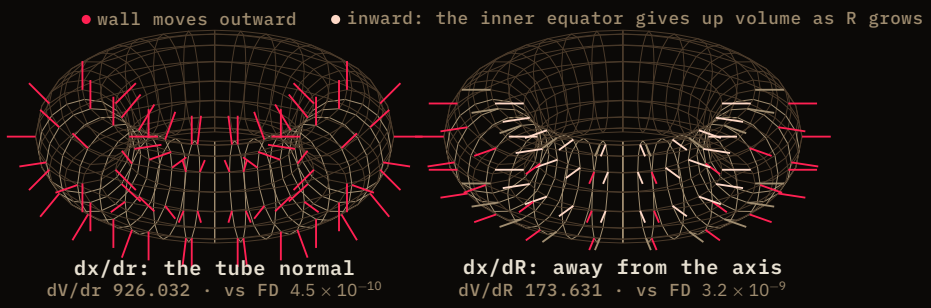



**FIG 5** **The torus's two design velocities** on its own mesh, front half. Growing the minor radius moves every node along the tube normal. Growing the major radius moves every node away from the axis, so the wall moves outward on the outer equator and inward on the inner one, which is why $dV/dR = 2\pi^2 r^2$ is smaller than $dV/dr = 4\pi^2 Rr$ here. Values are the seam's, on the 24-segment mesh of `make_torus(8, 3)`.

## 5 Reverse mode and optimization

Node velocities are linear in the seed values. For each node there is a fixed linear map from the per-surface seed slots (a translation in $\mathbb{R}^3$ and one slot per scalar field) to $\dot{x}_i$. Reverse mode is the transpose of these maps contracted with $\partial J/\partial x$:

$$\frac{\partial J}{\partial s} = \sum_i A_{is}^\top \frac{\partial J}{\partial x_i}, \quad \frac{dJ}{d\theta_k} = \sum_s \langle \frac{\partial J}{\partial s}, \mathrm{seed}_k(s) \rangle.$$

One `evaluate_with_pullback` computes the per-surface cotangents; each parameter then costs a few dot products. No tape or graph is recorded. The transposed vertex

solve runs the same Gram–Schmidt elimination carrying each coefficient as a linear functional of the rows, and each row's dependence on its seeds is read off by calling the forward row constructor with unit seeds, so forward and reverse mode cannot disagree about a row.

The optimizer is L-BFGS with memory 8[27], box projection and an Armijo line search on the projected step. The objective is smooth only within a topology class, so three rules keep bad curvature out of the memory: pairs with $s \cdot y \le 10^{-12}\|s\|\|y\|$ are dropped; a pair is dropped when the line search that produced it had to shrink past a `TopologyChanged` or `CorrespondenceLost` error, since its $y$ mixes two branches; and a direction that is not a descent direction clears the memory and falls back to projected steepest descent. Each iterate rebuilds the part and captures a fresh plan.

## 6 Seeding synthesis and documents

Writing a seeding by hand ("this radius moves these twenty surfaces, each at radius rate 1 with a center retreat of $(\pm 1, \pm 1, 0)$") is error-prone on real parts. `synthesize_seeding` derives it from the build function. It rebuilds at $\theta \pm h$, matches each base surface to its counterparts by frame-invariant geometric identity (a plane by its normal up to sign and its offset; a cylinder by its axis line and radius; a sphere by center and radius), and reads the field velocities off central differences of the matched fields. Surface fields, unlike mesh nodes, have no combinatorial structure, so this difference is accurate to $O(h^2)$. Only observable components are kept: in-plane drift of a plane's origin and axial drift of a cylinder's center are gauge and are projected out, while a cone's apex slide is observable because it changes the radius at every height. Everything downstream of the seeding remains exact.

`document_parameter_gradient` applies this to a `.vcad` document: a named parameter is bound, the document is evaluated to one BRep per part, the seeding is synthesized, and each solid part returns $d(V, m, c, I)/d\theta$. Evaluations at $\theta \pm h$ are checked to produce the same number of parts, so a parameter value that adds or removes a part errors instead of returning a gradient.

## 7 Gradients through rigid-body rollouts

In a rollout of an articulated rigid-body system without contact, the geometry of body $b$ enters the dynamics only through its ten mass-property scalars $p_b = (m, c, I)$. The chain rule is exact:

$$\frac{dJ}{d\theta} = \sum_b \frac{\partial J}{\partial p_b} \cdot \frac{dp_b}{d\theta} + \sum_b \text{pullback}_b\left(\frac{\partial J}{\partial x}\right).$$

The first factor of the first term, $\partial J/\partial p_b$, comes from a discrete adjoint over semi-implicit Euler in phyz[24]. Its within-step Jacobian columns are themselves exact: the articulated-body algorithm, forward kinematics and contact law are generic over the scalar type and are evaluated on dual numbers, one lane per column. The second factor, $dp_b/d\theta$, is the seam's mass-property derivative. The second term is the contact channel: phyz's differentiable rollout applies a penalty force $f_z = \max(0, k\delta - cv_z)$ at

every vertex of the body's frozen-plan mesh below the ground, and its adjoint returns $\partial J/\partial x$ on those vertices, which the seam's reverse mode turns into $dJ/d\theta$ (Section 5). Vertices reach the dynamics only through each body's six-component contact wrench, so the adjoint prices the wrench cotangent once per body and contracts it against each vertex's local wrench Jacobian rather than carrying $3N$ dual lanes. A rollout that the structured specification cannot express (state feedback, multi-DOF joints) falls back to central differences on the ten scalars, which needs about twenty re-simulations per body and no CAD rebuild.

## 8 The seam as an operator node

Viewed from outside, everything in Section 3 to Section 7 is a small set of operators, each with a forward map and an exact vector–Jacobian product:

- **SeamEval**: $\theta \mapsto x$, node positions of the frozen plan; VJP by the transposed seam (Section 5). It fails, rather than returning a value, outside its topology class.
- **MassProps**: $x \mapsto p$, the ten mass-property scalars per body; VJP from the divergence-theorem gradients.
- **Rollout**: $(p, x) \mapsto J$, a phyz trajectory with penalty contact on the seam mesh; VJP from the discrete adjoint, returning both $\partial J/\partial p$ and the contact cotangent $\partial J/\partial x$.

In vcad today these three are composed by hand: `adjoint_gradient_core` in `vcad-kernel-physics` runs a nominal seam pass per body, computes mass properties, runs one phyz adjoint pass and contracts the two channels. That function is a three-node operator graph written out manually, and every rollout result in Table 3 comes from it.

The architecture we are building makes that graph general. Engineering physics in vcad (thermal and electromagnetic time steps, finite-element assembly, sparse matrix–vector products) has the shape of tang comprehensions: affine views and a map for a stencil step, a gather view with a scatter-add monoid for assembly. What a comprehension IR cannot express is "solve this system" or "run this simulation" as a single step with a known gradient, so the one new construct is an opaque operator node that runs native code forward and takes a user-supplied adjoint backward: `LinearSolve` (adjoint: one more solve with $A^\top$), `NonlinearSolve`, a checkpointed `TimeLoop`, phyz's `Rollout` and the `SeamEval` of this paper. ABA and contact stay native Rust inside `Rollout`; they are sequential recursions that already have a correct adjoint and gain nothing from being traced.

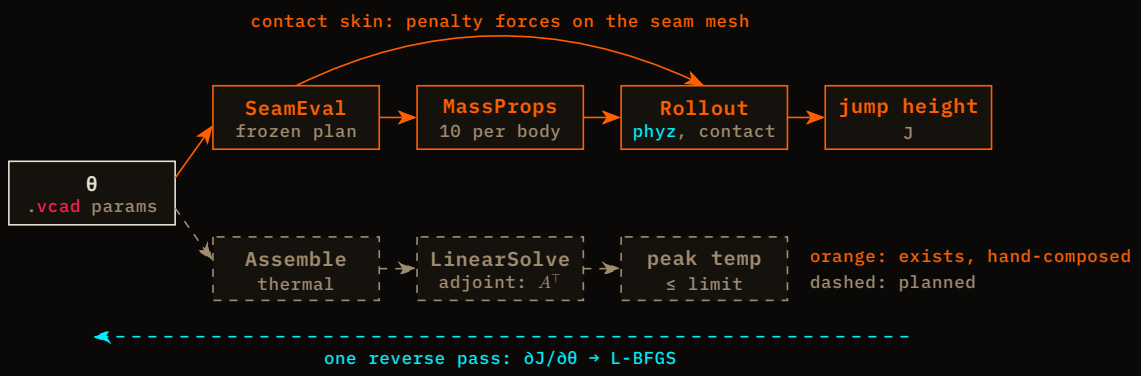



**FIG 6 The planned showpiece**, not a result of this paper. CAD parameters of a leg (wall thickness, fillet radius, foot pad) pass through SeamEval, MassProps and a phyz contact rollout to jump height, with a motor housing's steady-state temperature as a coupled constraint through assembly and a linear solve. One reverse pass gives $\partial J/\partial\theta$ for L-BFGS. The orange path exists today, composed by hand (Table 3); the thermal branch and the general graph are planned.

The plan is staged so that each step has a numeric gate: one tang version across vcad, phyz and tang; a sparse conjugate-gradient solver written in the IR and `LinearSolve` proved on steady thermal against vcad's hand adjoint to $10^{-12}$; thermal and FEA assembly as comprehensions on GPU; `Rollout` and `SeamEval` as nodes, matching today's hand-wired numbers to round-off before the hand wiring is deleted; coupled thermal stress through one graph; and Revolve-style checkpointing for long rollouts and transients. None of these gates has passed at the time of writing, and no result in Section 9 depends on them.

## 9 Results

All results come from vcad's gate tests at commit `d10f279d96`, run in the release profile on a single thread (12th Gen Intel(R) Core(TM) i9-12900K, rustc 1.98.0 (88d9e12ae 2026-08-18)). Each test asserts its bound; the values below are what the tests measured (Section 11). "FD" is a central difference with $h = 10^{-6}$ in the parameter unless stated: the part is rebuilt at $\theta \pm h$ and re-evaluated under the same frozen plan, so FD and the seam differentiate the same discrete object. Relative errors are taken against the larger of the two magnitudes.

## 9.1 Validation against finite differences

**TABLE 1 First-order volume gradients against finite differences of a full rebuild.** Node-wise is the worst relative error of $dx_i/d\theta$ over all mesh nodes. Torus rows list the FD value; the seam agrees to the error shown. Every row passes vcad's $10^{-6}$ gate. The filleted cube moves 20 surfaces with one parameter. `data/measured.tsv`.

| MODEL | θ | DV/Dθ (SEAM) | VS FD | NODE-WISE |
|---|---|---|---|---|
| Extrusion | depth | 12 | $8.3 \times 10^{-12}$ | $2.9 \times 10^{-11}$ |
| Box – cylinder | hole r | −78.499 | $6.2 \times 10^{-10}$ | $6.6 \times 10^{-10}$ |
| Cylinder | height | 77.646 | $2.9 \times 10^{-9}$ | $3 \times 10^{-10}$ |
| Filleted cube | fillet r | −72.807 | $9.5 \times 10^{-9}$ | $3.2 \times 10^{-9}$ |
| Filleted cube | edge a | 293.665 | $8.2 \times 10^{-10}$ | – |
| Cone frustum | half-angle | 3449.54 | $5.9 \times 10^{-12}$ | $9.6 \times 10^{-11}$ |
| Cone frustum | height | 27.952 | $4.7 \times 10^{-9}$ | $1.8 \times 10^{-7}$ |
| Torus | minor r | 926.03 | $4.5 \times 10^{-10}$ | $1.4 \times 10^{-9}$ |
| Torus | major R | 173.63 | $3.2 \times 10^{-9}$ | $1.3 \times 10^{-9}$ |

Where a discrete closed form exists the seam meets it to rounding. The hole's mesh cross-section is the inscribed 112-gon, so $dV/dr = -N \sin(2\pi/N) rt$; the seam matches it to $9.1 \times 10^{-16}$. The cone's half-angle derivative matches the prismatoid formula to $1.3 \times 10^{-16}$, the cylinder-height derivative the N-gon cap area to $1.8 \times 10^{-16}$, and the extrusion's $dV/dd$ the cross-section area exactly (0). The continuum shape derivatives differ by the polygonization error and no more: $5.2 \times 10^{-4}$ for the hole (bounded by $x^2/6$ with $x = 2\pi/N$), and 6.7% for the fillet's Minkowski-sum derivative at 16 segments per quarter-blend.

The mass-property derivatives behave the same way: $dI_{zz}/dd$ of the extrusion meets its closed form to $5.7 \times 10^{-16}$ and FD to $3.2 \times 10^{-10}$ the cylinder's inertia derivative with respect to radius meets FD to $8.7 \times 10^{-11}$.

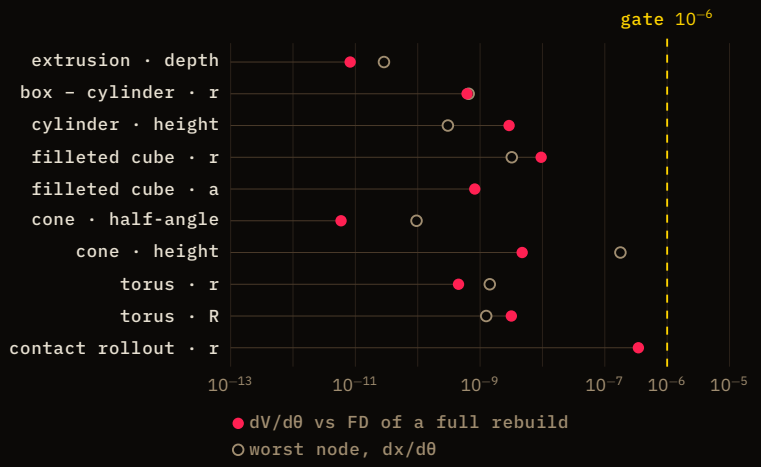



**FIG 7 Every first-order gate on one axis.** Filled: the seam's $dV/d\theta$ (or $dJ/d\theta$ for the contact rollout) against a central difference of a full rebuild. Open: the worst node-wise $dx/d\theta$. All sit two or more orders below vcad's $10^{-6}$ gate. `data/measured.tsv`.

## 9.2 Reverse mode, synthesis and second order

**TABLE 2 Reverse mode, synthesized seedings and second derivatives.** Reverse and forward mode use the same rows in a different order and agree to rounding. Synthesized seedings carry the $O(h^2)$ error of the field differences. The filleted cube's second derivative is checked with a wider FD step because re-capturing a filleted solid at $r \pm h$ adds about $10^{-4}$ of correspondence jitter (Section 10). The Gauss–Newton comparison is on the five-parameter box near its optimum, where the dropped residual term vanishes. `data/measured.tsv`.

| CHECK | VALUE | REL. ERROR |
|---|---|---|
| Reverse vs forward, hole dV/dr | −78.4986 | $9.1 \times 10^{-16}$ |
| Reverse vs forward, fillet dV/dr | −72.8072 | $2.9 \times 10^{-15}$ |
| Synthesized vs hand seeding, fillet dV/dr | −72.8072 | $1.8 \times 10^{-9}$ |
| Synthesized vs hand seeding, edge dV/da | 293.665 | $7.5 \times 10^{-10}$ |
| Synthesized, cone/torus composite vs FD | 926.03 | $3.1 \times 10^{-10}$ |
| Reverse on synthesized seeding vs forward | −72.8072 | $3.9 \times 10^{-16}$ |
| d²V/dr², hole vs closed form | −31.399 | $2.6 \times 10^{-15}$ |
| d²V/dα², cone vs closed form | 16385.3 | $2.2 \times 10^{-16}$ |
| d²V/dα², cone vs FD of dV/dα | | $4.5 \times 10^{-11}$ |
| d²V/dr², torus minor vs FD of dV/dr | 308.68 | $1.7 \times 10^{-6}$ |
| d²V/dθ², torus with r = θ² (chain rule) | 5556.2 | $2.1 \times 10^{-15}$ |
| d²V/dr², filleted cube vs FD (h = 10□³) | −40.72 | $1.2 \times 10^{-3}$ |
| Gauss–Newton vs full Hessian (Frobenius) | | $5.3 \times 10^{-4}$ |

The cone gate is the first in which node accelerations are nonzero: opening the half-angle moves every rim vertex along a curved path, and the nested-dual lift and the cone's curvature rows reproduce the closed form to rounding.

## 9.3 L-BFGS on a five-parameter part

The test part is a box of dimensions $(s_x, s_y, s_z)$ with all edges filleted at radius $r$ and a centered through-hole of radius $r_h$: 6 planes, 12 blend cylinders, 8 corner spheres and a hole wall, 916 mesh nodes. The objective is the squared relative miss of five quantities (volume, the three centroid coordinates and $I_{zz}$) against their values at $\theta^* = (10, 12, 8, 1.8, 2.2)$ mm, starting from $\theta_0 = (8, 14, 6, 1.2, 1.6)$ mm. The five-by-five Jacobian of the quantities has determinant 35457 and smallest pivot 0.34, so all five parameters are locally identifiable.

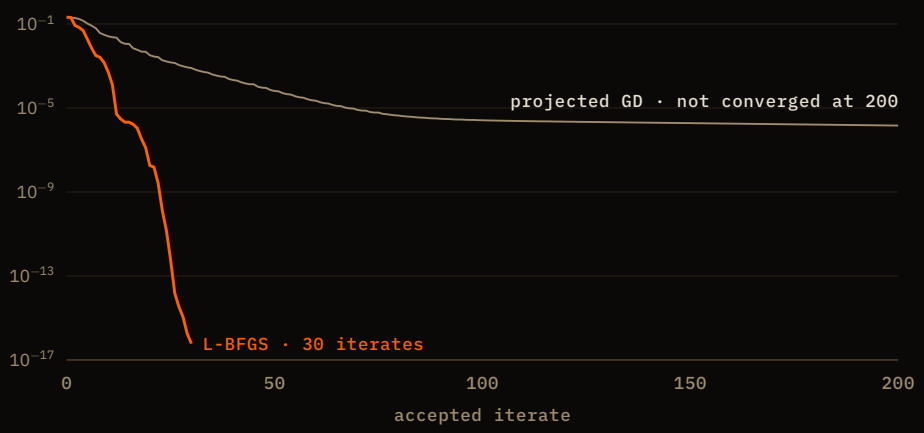



**FIG 8 L-BFGS against projected gradient descent**, same gradients (reverse mode), same line search. L-BFGS stops on the gradient tolerance after 30 iterations (33 objective evaluations) with $\|\theta - \theta^*\|_\infty = 2.9 \times 10^{-7}$. Gradient descent reaches its 200-iteration limit (395 evaluations) at $\|\theta - \theta^*\|_\infty = 6.4 \times 10^{-2}$: steepest descent zig-zags between the dimension and radius directions, whose scales differ. `data/measured.tsv`.

Reverse and forward gradients agree per component to at most $4.2 \times 10^{-16}$. Per iterate, the shared rebuild and capture take 83 ms; differentiation takes 1.27 ms in forward mode (five seam passes) and 0.6 ms in reverse mode (one seam pass, one pullback, five dot products). The rebuild dominates at this size; the differentiation cost is the part that grows with the number of parameters, linearly in forward mode and by one dot product per parameter in reverse mode.

A finite-difference check of the same gradient shows why finite differences are a poor substitute on filleted parts. For the hole radius and $s_z$ FD agrees to $1.9 \times 10^{-9}$ and $2.6 \times 10^{-9}$, but for $s_x$ and the fillet radius the agreement is only $2.5 \times 10^{-5}$ and $1.4 \times 10^{-4}$. The error is in FD, not in the seam. Rebuilding a filleted solid under a plan captured elsewhere carries the fillet kernel's frame and correspondence noise, while reverse and forward mode, two independent implementations, agree to rounding.

## 9.4 A fillet radius and a flywheel by gradient descent

Projected gradient descent on the fillet radius alone recovers the radius whose rounded cube hits a target volume, $r^* = 2.2$ mm from $r_0 = 1$ mm, landing at 2.2 mm in 17 accepted iterates.

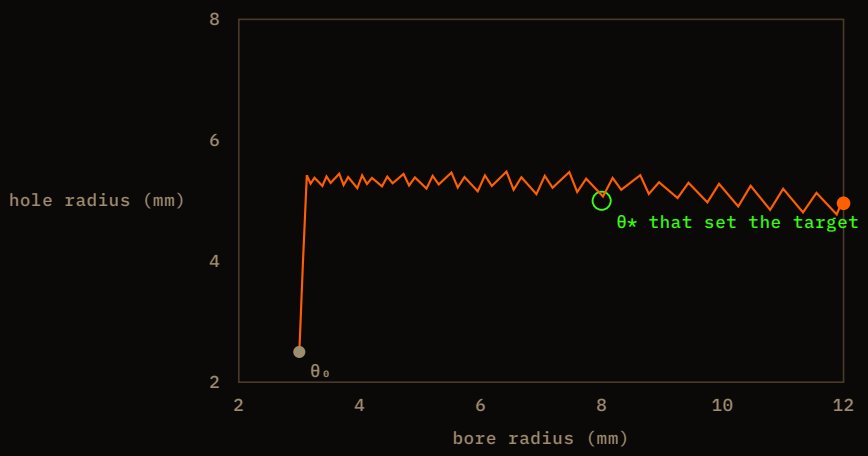



**FIG 9 A flywheel design brief.** A disc (radius 40 mm, thickness 10 mm) with a center bore and four lightening holes on a 25 mm bolt circle; θ = (bore radius, hole radius). Objective: mass over a reference mass plus 100 · the squared relative miss of a target spin inertia, the inertia of θ* = (8, 5). Projected gradient descent, each iterate rebuilding the part through five Booleans and capturing a fresh plan. The bounds keep the holes inside the disc and clear of each other, so the topology does not change. Descent is monotone over 78 accepted iterates, ending at θ = (12, 4.96) mm with the bore on its upper bound: θ* only sets the target inertia, and a larger bore reaches it with less mass. The zig-zag in hole radius is steepest descent on an anisotropic objective; the analytic gradient matched FD at $\theta_0$ to $1.4 \times 10^{-9}$. `data/measured.tsv`.

## 9.5 A named parameter of a document

A `.vcad` document whose cylinder radius is bound to the named parameter `r` differentiates end to end with no hand seeding: $dV/dr = 501.848$ against the N-gon closed form 501.848 ($2.3 \times 10^{-12}$), and $dI_{zz}/dr$ agrees with a document-rebuild FD to $4.1 \times 10^{-11}$. A document authored in vcad's loon language and one whose parameter is declared with `defparam` reach the same closed form to $2.3 \times 10^{-12}$ and $2.3 \times 10^{-12}$.

## 9.6 Shape through dynamics and contact

**TABLE 3 Gradients of rollouts with respect to a CAD radius**: J in rad/s, rad and m; θ in mm. The reference rebuilds the CAD at $\theta \pm h$, re-tessellates, recomputes mass properties and re-simulates, with contact where the rollout has it. The adjoint's pendulum is the same cylinder re-expressed for the adjoint's specification (revolute about the body x axis instead of a rotated inertia), so its value differs from the first pendulum's. The adjoint agrees with the finite-difference path on $p$ to $1.2 \times 10^{-10}$ (flywheel) and $4.1 \times 10^{-10}$ (pendulum). `data/measured.tsv`.

| ROLLOUT (θ = CYLINDER RADIUS) | DJ/DΘ | VS END-TO-END FD |
|---|---|---|
| Flywheel spin-up, $J = \omega(T)$, FD on p | −23.6544 | $5 \times 10^{-8}$ |
| Gravity pendulum, $J = q(T)$, FD on p | −0.22795 | $8.3 \times 10^{-10}$ |
| Flywheel, adjoint | −23.6544 | $5 \times 10^{-8}$ |
| Pendulum on body-x revolute, adjoint | 0.19138 | $1.2 \times 10^{-9}$ |
| Cylinder resting on its side, contact adjoint | $9.82 \times 10^{-4}$ | $3.5 \times 10^{-7}$ |

The contact case is the shape-through-contact co-design problem in its smallest form. A CAD cylinder lies on its side on a vertical prismatic joint and settles onto the ground; $J$ is its settled height. Growing the radius moves the contact line down the body by exactly $d\theta$, which alone would give $dJ/d\theta = 10^{-3}$ m per mm (the surface channel), and adds mass that sinks the body further into the penalty spring (the mass

channel, negative). The two channels have opposite signs. The gradient, $9.82 \times 10^{-4}$, is 1.8% below the surface channel alone, and matches a rebuild-and-resimulate FD to $3.5 \times 10^{-7}$ with no finite difference anywhere in the analytic chain. The test also asserts that doubling the density lowers the gradient, so the mass channel contributes to the result.

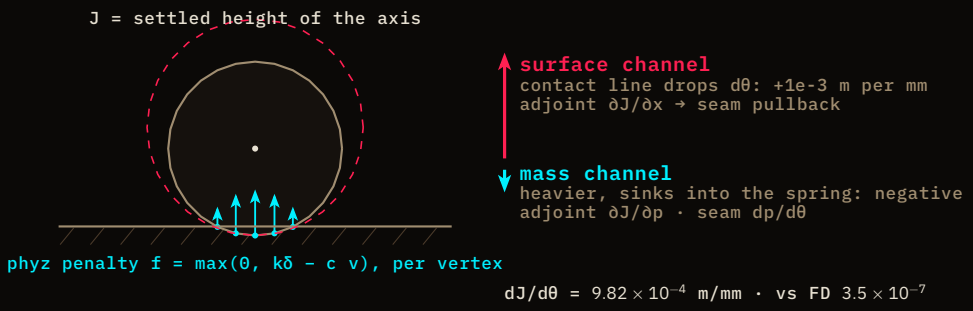



**FIG 10 Shape through contact.** A CAD cylinder (θ = radius) rests on its side on phyz's per-vertex penalty springs; $J$ is its settled height. Growing the radius raises the body through the surface channel and sinks it through the mass channel. Both factors are exact, so the chain has no finite difference in it. `data/measured.tsv`.

## 9.7 Pixels to CAD parameters: a shared benchmark with irys

The seam's reverse mode accepts any cotangent on the frozen plan's nodes, including one from a renderer. With the irys differentiable path tracer (TODO: cite the irys paper once it has an arXiv identifier) we ran a shared benchmark on a real robot part: the left shoulder-pitch output plate of the rose arm, a 3 mm rounded plate with a 49 mm bearing bore and twelve 4.4 mm bolt holes. θ is the bore diameter. The frozen plan has 4942 nodes and 9932 triangles at 32 circle segments, with a synthesized seeding. irys renders the plan's mesh as it is, without re-tessellating, against an empty background at 160 × 128, so the whole derivative comes from visibility edges; its edge sampler returns $\partial \mathcal{L}/\partial x$ on every plan node for a fixed smooth image loss.

**TABLE 4 Shared rose-arm benchmark**, measured by the irys session at irys `103e3da` and vcad `d10f279` rose `832192f`; not regenerated by `repro.sh`. The pullback's Monte Carlo standard error over 8 edge-sampling seeds is ±0.0226. The render FD itself moves by about 0.5% between $h = 0.02$ and 0.1 mm, so the pullback agrees with it within the FD's own error.

| PATH TO DL/DΘ | VALUE | VS PULLBACK |
|---|---|---|
| Seam pullback (reverse mode) | +40.16864 | – |
| Seam forward mode, $\sum \partial \mathcal{L}/\partial x \cdot \dot{x}$ | +40.16864 | $1.1 \times 10^{-15}$ |
| vcad FD node velocities ($h = 10^{-5}$ mm) | +40.17028 | $4.3 \times 10^{-5}$ |
| End-to-end render FD ($h = 0.1$ mm) | +40.0144 | $3.9 \times 10^{-3}$ |

The benchmark also shows why an end-to-end finite difference through a real part is a poor reference. A true rebuild at $\theta \pm h$ fails with `CorrespondenceLost` for $h \geq 2 \times 10^{-3}$ mm and with `TopologyChanged` at 0.05 mm, because the bore rim's arc count is sag-adaptive and drops from 4942 to 4940 vertices; at $h = 5 \times 10^{-4}$ mm the render FD is dominated by quadrature noise. The end-to-end check therefore moves the plan's nodes along their FD velocities rather than rebuilding. The seam has no such step-size window: its pullback is exact for the frozen mesh, and the renderer's sampling error is the only noise left.

### 9.8 Cost

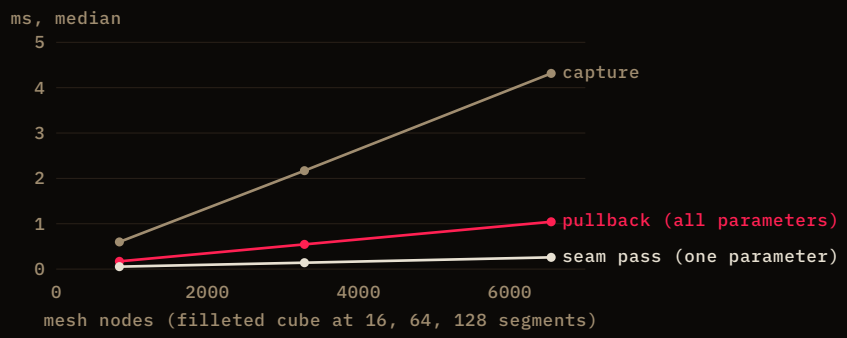



**FIG 11 Per-call cost on the filleted cube** at three blend resolutions, release build, median. Capture, one forward seam pass and one pullback are all linear in the number of mesh nodes; the pullback prices every parameter at once. `data/measured.tsv`.

## 10 Discussion

**Why freeze the tessellation.** The alternative, differentiating the kernel's code, has to give some answer at every branch: which face a Boolean keeps, how a fillet orders its blends, how many segments a circle gets. Those answers are piecewise constant in $\theta$, so their derivatives are zero almost everywhere and undefined at the switches, and they are entangled with the nondeterminism we observed in the Boolean and fillet code. Freezing the plan turns the discrete state into an object the seam can check. Inside the plan's topology class the derivative is exact, and outside it the seam returns an error.

**What the gradient is a gradient of.** The seam's derivative is that of the mesh vcad produces, which is the object a mesh-based simulation consumes. For mass properties at practical resolutions the gap to the continuum is small (Table 1); for objectives that depend on fine surface detail it may not be, and the remedy is resolution.

**Finite differences are not a reliable reference on filleted solids.** On every plane, cylinder, cone and torus gate, FD and the seam agree to $10^{-8}$ or better. On filleted solids FD at $h = 10^{-6}$ degrades to $10^{-4}$–$10^{-5}$ for some parameters (Figure 8) because a rebuild at $\theta \pm h$ carries correspondence jitter of that order; we validated those cases by agreement between independent forward and reverse implementations and by FD with a wider step.

**From a seam to a graph.** The natural next result is the showpiece of Figure 6: the rollout path of Table 3 with the thermal branch added, through operator nodes rather than hand composition, and validated by the same rebuild-and-resimulate differences. Because the seam returns an error outside its topology class instead of a wrong value, a graph that contains it never receives a wrong gradient from it.

**Remaining gaps.** Cone-tangent-to-plane tangency rows are not implemented, because no gate model has produced one. Fillets whose corners are asymmetric or meet three blends fall back to the kernel's older pipeline. The contact model is frictionless; friction's kink at sticking would need smoothing. Running costs in

the rollout objective, multi-DOF joints and state-feedback control are mechanical extensions of the adjoint recursion that nothing has needed yet.

## 11 Reproducibility

`paper/vcad/repro/repro.sh` clones vcad at commit `d10f279d9693d307c7faa7289ec6490b12f0bd77` (its lockfile pins phyz, loon and tang), applies `report.patch`, runs the gate tests listed above with `--nocapture`, and writes `data/measured.tsv`. The patch is generated by `instrument.py` and only adds `eprintln!` lines after statements that already compute a measured quantity, so each gate still asserts its own bound and the printed value is the one it checks. This document reads every measured number, table cell and plotted point from that file; rebuilding the PDF after a run updates them. The run used for this draft took place on 2026-11-07T00:59Z on Linux 6.8.0-147-generic x86_64.

## 12 Conclusion

Exact parameter gradients through a production BRep kernel are attainable without making the kernel differentiable. Cutting the chain at a frozen tessellation, and differentiating only the geometric map from parameters to mesh nodes, yields derivatives that are exact for the mesh, cover the parameters designers edit (including fillet radii), compose with reverse mode, second order and L-BFGS, and carry through to rigid-body dynamics and contact. They do not cross a change of topology. The kernel reports that case as an error. In the architecture we are building, the seam is the geometry operator in a graph where vcad's physics is written once as math and tang derives the kernels and gradients for every device. vcad, including the seam, is open source.

## References

1. Gilbert, E. G., Johnson, D. W. & Keerthi, S. S. A fast procedure for computing the distance between complex objects in three-dimensional space. *IEEE Journal on Robotics and Automation* **4,** 193–203 (1988).
2. Kato, H., Ushiku, Y. & Harada, T. Neural 3D Mesh Renderer. in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2018).
3. Liu, S., Li, T., Chen, W. & Li, H. Soft Rasterizer: A Differentiable Renderer for Image-based 3D Reasoning. in *IEEE International Conference on Computer Vision (ICCV)* (2019).
4. Li, T.-M., Aittala, M., Durand, F. & Lehtinen, J. Differentiable Monte Carlo Ray Tracing through Edge Sampling. *ACM Transactions on Graphics* **37,** (2018).
5. Worchel, M. & Alexa, M. Differentiable Rendering of Parametric Geometry. *ACM Transactions on Graphics* **42,** 232:1–232:18 (2023).

6. Park, J. J., Florence, P., Straub, J., Newcombe, R. & Lovegrove, S. DeepSDF: Learning Continuous Signed Distance Functions for Shape Representation. in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2019).
7. Vicini, D., Speierer, S. & Jakob, W. Differentiable Signed Distance Function Rendering. *ACM Transactions on Graphics* **41,** (2022).
8. Sharma, G., Goyal, R., Liu, D., Kalogerakis, E. & Maji, S. CSGNet: Neural Shape Parser for Constructive Solid Geometry. in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2018).
9. Du, T. *et al.* InverseCSG: Automatic Conversion of 3D Models to CSG Trees. *ACM Transactions on Graphics* **37,** (2018).
10. Cascaval, D. *et al.* Differentiable 3D CAD Programs for Bidirectional Editing. *Computer Graphics Forum* **41,** 309–323 (2022).
11. Michel, É. & Boubekeur, T. DAG Amendment for Inverse Control of Parametric Shapes. *ACM Transactions on Graphics* **40,** (2021).
12. Schulz, A. *et al.* Interactive Design Space Exploration and Optimization for CAD Models. *ACM Transactions on Graphics* **36,** (2017).
13. Banović, M. *et al.* Algorithmic differentiation of the Open CASCADE Technology CAD kernel and its coupling with an adjoint CFD solver. *Optimization Methods and Software* **33,** 813–828 (2018).
14. Haftka, R. T. & Grandhi, R. V. Structural shape optimization—a survey. *Computer Methods in Applied Mechanics and Engineering* **57,** 91–106 (1986).
15. Sokołowski, J. & Zolésio, J.-P. *Introduction to Shape Optimization: Shape Sensitivity Analysis*. (Springer, 1992).
16. Agarwal, D. *et al.* Parametric design velocity computation for CAD-based design optimization using adjoint methods. *Engineering with Computers* **34,** 225–239 (2018).
17. Hughes, T. J. R., Cottrell, J. A. & Bazilevs, Y. Isogeometric analysis: CAD, finite elements, NURBS, exact geometry and mesh refinement. *Computer Methods in Applied Mechanics and Engineering* **194,** 4135–4195 (2005).
18. Avila Belbute-Peres, F. de, Smith, K., Allen, K., Tenenbaum, J. & Kolter, J. Z. End-to-End Differentiable Physics for Learning and Control. in *Advances in Neural Information Processing Systems (NeurIPS)* (2018).
19. Geilinger, M. *et al.* ADD: Analytically Differentiable Dynamics for Multi-Body Systems with Frictional Contact. *ACM Transactions on Graphics* **39,** (2020).
20. Werling, K., Omens, D., Lee, J., Exarchos, I. & Liu, C. K. Fast and Feature-Complete Differentiable Physics for Articulated Rigid Bodies with Contact. in *Robotics: Science and Systems (RSS)* (2021).
21. Hu, Y. *et al.* DiffTaichi: Differentiable Programming for Physical Simulation. in *International Conference on Learning Representations (ICLR)* (2020).

22. Ha, S., Coros, S., Alspach, A., Kim, J. & Yamane, K. Joint Optimization of Robot Design and Motion Parameters using the Implicit Function Theorem. in *Robotics: Science and Systems (RSS)* (2017).
23. Xu, J. *et al.* An End-to-End Differentiable Framework for Contact-Aware Robot Design. in *Robotics: Science and Systems (RSS)* (2021).
24. Featherstone, R. *Rigid Body Dynamics Algorithms*. (Springer, 2008).
25. Griewank, A. & Walther, A. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*. (SIAM, 2008).
26. Mirtich, B. Fast and Accurate Computation of Polyhedral Mass Properties. *Journal of Graphics Tools* **1,** 31–50 (1996).
27. Liu, D. C. & Nocedal, J. On the limited memory BFGS method for large scale optimization. *Mathematical Programming* **45,** 503–528 (1989).